What private Really Means

Now that you know about self, you can cast a new light on Ruby's private keyword. Private methods are governed by a single simple rule: you cannot call a private method with an explicit receiver. In other words, every time you call a private method, it must be on the implicit receiver—self. Let's see a corner case:

```
class C
  def public_method
    self.private_method
  end

  private

  def private_method; end
end

C.new.public_method
```

```
NoMethodError: private method 'private_method' called [...]
```

You can make this code work by removing the self keyword.

This contrived example shows that private methods come from two rules working together: first, you need an explicit receiver to call a method on an object that is not yourself, and second, private methods can be called only with an implicit receiver. Put these two rules together, and you'll see that you can only call a private method on yourself. You can call this the "private rule."

You could find Ruby's private methods perplexing—especially if you come from Java or C#, where private behaves differently. When you're in doubt, go back to the private rule, and everything will make sense. Can object x call a private method on object y if the two objects share the same class? The answer is no, because no matter which class you belong to, you still need an explicit receiver to call another object's method. Can you call a private method that you inherited from a superclass? The answer is yes, because you don't need an explicit receiver to call inherited methods on yourself.

If you want to become a master of Ruby, you should always know which object has the role self at any given moment. In most cases, that's easy: just track which object was the last method receiver. However, there are two important special cases that you should be aware of. Let's look at them.

The Top Level

You just learned that every time you call a method on an object, that object becomes self. But then, who's self if you haven't called any method yet? You can run irb and ask Ruby itself for an answer:

```
self        # => main
self.class  # => Object
```